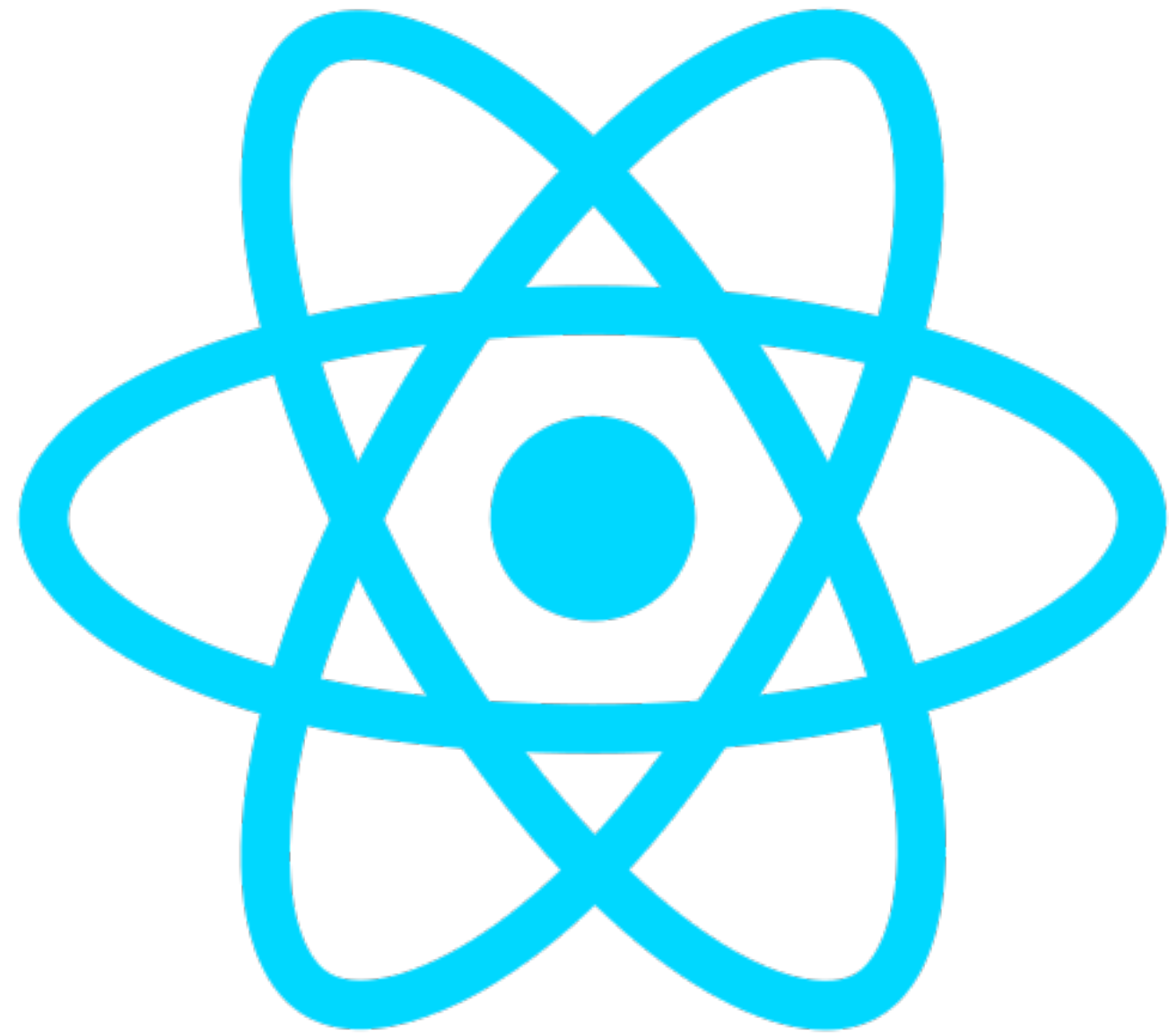




REACT NATIVE



Lecture 16: Working with device specific API's

Shubhang Sharma
Taraksh Goyal
Deepak Soni

Cognition 4.0
2025

UPES MTC



Device Specific API's

- These API's help you have control of your mobile device features and helps you use them.
- Example. Gyroscope , Camera , Location , Clipboard etc

Clipboard

- Provides access to an API for getting and setting the device clipboard.
- `Bunx expo install expo-clipboard || npx expo install expo-clipboard.`


```
import { useState } from 'react';
import { View, Text, Button, StyleSheet } from 'react-native';
import * as Clipboard from 'expo-clipboard';

export default function App() {
  const [copiedText, setCopiedText] = useState('');

  const copyToClipboard = async () => {
    await Clipboard.setStringAsync('hello world');
  };

  const fetchCopiedText = async () => {
    const text = await Clipboard.getStringAsync();
    setCopiedText(text);
  };

  return (
    <View style={styles.container}>
      <Button title="Click here to copy to Clipboard" onPress={copyToClipboard} />
      <Button title="View copied text" onPress={fetchCopiedText} />
      <Text style={styles.copiedText}>{copiedText}</Text>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
  },
  copiedText: {
    marginTop: 10,
    color: 'red',
  },
});
```

Battery

- A library that provides battery information for the physical device, as well as corresponding event listeners.
- `bunx expo install expo-battery || npx expo install expo-battery`
- `useBatteryLevel()` → The battery level of device
- `useBatteryState()` → The battery state of device
- `useLowPowerMode()` → If device is in Low Power Mode
- Docs: <https://docs.expo.dev/versions/latest/sdk/battery>

```
import                               from "expo-battery"
import { StyleSheet, Text, View } from "react-native";

export default function App() {
  const                               useBatteryLevel

  return (
    <View style={styles.container}>
      <Text>
        Current Battery Level:{" "}
        {batteryLevel !== null
          ? (batteryLevel * 100).toFixed(1) + "%"
          : "Loading..."}
      </Text>
      <Text>
        Raw Value: {batteryLevel !== null ? batteryLevel : "Loading..."}
      </Text>
    </View>
  )
}
```

```
const                               StyleSheet create
  container: {
    flex 1
    marginTop: 15,
    alignItems: "center",
    justifyContent: "center",
    backgroundColor: "#fff",
  },
}
```


Brightness

- Provides access to an API for getting and setting the screen brightness.
- `Bunx expo install expo-brightness || npx expo install expo-brightness.`

```
import           from 'react'
import { StyleSheet, View, Text } from 'react-native';
import * as Brightness from 'expo-brightness';

export default function App() {
  useEffect(() => {
    async
    const { status } = await Brightness.requestPermissionsAsync();
    if 'granted'
      Brightness.setSystemBrightnessAsync(1);

  return (
    <View style={styles.container}>
      <Text>                                </Text>
    </View>

  const           StyleSheet create
  container: {
    flex
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
```


Location

- Allows reading geolocation information from the device. Your app can poll for the current location or subscribe to location update events.
- `Bunx expo install expo-location || npx expo install expo-location.`

Camera

- Provides a React component that renders a preview of the device's front or back camera. The camera's parameters such as zoom, torch, and flash mode are adjustable. Using `CameraView`, you can take photos and record videos that are saved to the app's cache. The component is also capable of detecting bar codes appearing in the preview.
- `Bunx expo install expo-camera || npx expo install expo-camera.`

```
import { CameraView, CameraType, useCameraPermissions } from "expo-camera";
import { useState } from "react";
import { Button, StyleSheet, Text, TouchableOpacity, View } from "react-native";

export default function App() {
  const [facing, setFacing] = useState("back");
  const [permission, requestPermission] = useCameraPermissions();

  if (!permission) {
    // Camera permissions are still loading.
    return <View />;
  }

  if (!permission.granted) {
    // Camera permissions are not granted yet.
    return (
      <View style={styles.container}>
        <Text style={styles.message}>
          We need your permission to show the camera
        </Text>
        <Button onPress={requestPermission} title="grant permission" />
      </View>
    );
  }

  function toggleCameraFacing() {
    setFacing((current) => (current === "back" ? "front" : "back"));
  }

  return (
    <View style={styles.container}>
      <CameraView style={styles.camera} facing={facing}>
        <View style={styles.buttonContainer}>
          <TouchableOpacity style={styles.button} onPress={toggleCameraFacing}>
            <Text style={styles.text}>Flip Camera</Text>
          </TouchableOpacity>
        </View>
      </CameraView>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: "center",
  },
  message: {
    textAlign: "center",
    paddingBottom: 10,
  },
  camera: {
    flex: 1,
  },
  buttonContainer: {
    flex: 1,
    flexDirection: "row",
    backgroundColor: "transparent",
    margin: 64,
  },
  button: {
    flex: 1,
    alignSelf: "flex-end",
    alignItems: "center",
  },
  text: {
    fontSize: 24,
    fontWeight: "bold",
    color: "white",
  },
});
```


Keeping Screen Awake

- Provides a React hook that prevents the screen from sleeping and a pair of functions to enable this behavior imperatively.
- Bunx expo install expo-keep-awake || npx expo install expo-keep-awake.

```
import { useKeepAwake } from 'expo-keep-awake';
import React from 'react';
import { Text, View } from 'react-native';

export default function KeepAwakeExample() {
  useKeepAwake();
  return (
    <View style={{ flex: 1, alignItems: 'center', justifyContent: 'center' }}>
      <Text>This screen will never sleep!</Text>
    </View>
  );
}
```

Device Details

- Provides access to system information about the physical device, such as its manufacturer and model.

```
import { Text, View } from "react-native";
import * as Device from "expo-device";

export default function App() {
  return (
    <View style={{ flex: 1, justifyContent: "center", alignItems: "center" }}>
      <Text>
        {Device.manufacturer}: {Device.modelName}
      </Text>
    </View>
  );
}
```

Document Picker

- Provides access to the system's UI for selecting documents from the available providers on the user's device.
- `Bunx expo install expo-document-picker // npx expo install expo-document-picker`
-


```
import React, { useState } from "react";
import { Text, View, Button } from "react-native";
import * as DocumentPicker from "expo-document-picker";

export default function App() {
  const [file, setFile] = useState(null);

  const pickDocument = async () => {
    const result = await DocumentPicker.getDocumentAsync({});
    if (!result.canceled) {
      console.log(result);
      setFile(result);
    }
  };

  return (
    <View
      style={{
        flex: 1,
        justifyContent: "center",
        alignItems: "center",
        backgroundColor: "#fff",
      }}
    >
      <Button title="Pick a document" onPress={pickDocument} />
      {file && (
        <Text>
          Picked file: {file["assets"][0].name} ({file["assets"][0].size} bytes)
        </Text>
      )}
    </View>
  );
}
```

File System

- Provides access to a file system stored locally on the device. It is also capable of uploading and downloading files from network URLs.
- `Bunx expo install expo-file-system // npx expo install expo-file-system`


```
import React, { useState } from "react";
import { View, TextInput, Button, Text, Image, StyleSheet } from "react-native";
import * as FileSystem from "expo-file-system";

const gifDir = FileSystem.cacheDirectory + "giphy/";
const gifUrl = (id) => `https://media1.giphy.com/media/${id}/200.gif`;
const gifFileUri = (id) => `${gifDir}gif_${id}_200.gif`;

const ensureDir = async () => {
  if (!(await FileSystem.getInfoAsync(gifDir)).exists) {
    await FileSystem.makeDirectoryAsync(gifDir, { intermediates: true });
  }
};

const getGif = async (id) => {
  await ensureDir();
  const uri = gifFileUri(id);
  if (!(await FileSystem.getInfoAsync(uri)).exists) {
    await FileSystem.downloadAsync(gifUrl(id), uri);
  }
  return uri;
};

export default function GifInstaller() {
  const [gifId, setGifId] = useState("");
  const [status, setStatus] = useState("");
  const [gifUri, setGifUri] = useState(null);

  const handleInstall = async () => {
    if (!gifId.trim()) return setStatus("Enter a GIF ID.");
    setStatus("Installing...");
    try {
      setGifUri(await getGif(gifId.trim()));
      setStatus("GIF ready!");
    } catch {
      setStatus("Failed to load GIF.");
    }
  };

  return (
    <View style={styles.container}>
      <Text style={styles.label}>GIF ID:</Text>
      <TextInput
        style={styles.input}
        value={gifId}
        onChangeText={setGifId}
        placeholder="e.g. l0MYt5jPR6QX5pnqM"
      />
      <Button title="Load GIF" onPress={handleInstall} />
      {status ? <Text style={styles.status}>{status}</Text> : null}
      {gifUri ? <Image source={{ uri: gifUri }} style={styles.gif} /> : null}
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, padding: 20, justifyContent: "center" },
  label: { fontSize: 16, marginBottom: 8 },
  input: {
    borderWidth: 1,
    borderColor: "#ccc",
    padding: 8,
    marginBottom: 12,
    borderRadius: 4,
  },
  status: { fontSize: 16, marginTop: 12 },
  gif: { width: 200, height: 200, marginTop: 20 },
});
```

SENSORS

Accelerometer

- Provides access to the device accelerometer sensor(s) and associated listeners to respond to changes in acceleration in three-dimensional space, meaning any movement or vibration.
- Bunx expo install expo-sensors || npx expo install expo-sensors

```
const [{ x, y, z }, setData] = useState({  
  x: 0,  
  y: 0,  
  z: 0,  
});
```

```
Accelerometer.addListener(setData))
```


Barometer

- provides access to the device barometer sensor to respond to changes in air pressure, which is measured in hectopascals (hPa).
- `Bunx expo install expo-sensors || npx expo install expo-sensors`
- `const [{ pressure, relativeAltitude }, setData] = useState({ pressure: 0, relativeAltitude: 0 });`
- `Barometer.addListener(setData)`

Gyroscope

- Provides access to the device's gyroscope sensor to respond to changes in rotation in three-dimensional space.
- Bunx expo install expo-sensors || npx expo install expo-sensors
- ```
const [{ x, y, z }, setData] = useState({
 x: 0,
 y: 0,
 z: 0,
});
```
- ```
Gyroscope.addListener(gyroscopeData => {  
  setData(gyroscopeData);  
})
```

Sharing

- Allows you to share files directly with other compatible applications.
- `Bunx expo install expo-sharing || npx expo install expo-sharing`


```
import { View, Button } from "react-native";
import * as Sharing from "expo-sharing";
import * as FileSystem from "expo-file-system";

const FileShareExample = () => {
  const onShare = async () => {
    if (!(await Sharing.isAvailableAsync())) {
      console.log("Sharing is not available on this device");
      return;
    }
    const fileUri = FileSystem.cacheDirectory + "message.txt";
    await FileSystem.writeAsStringAsync(fileUri, "Hello from my app!");
    await Sharing.shareAsync(fileUri, {
      dialogTitle: "Share",
      mimeType: "text/plain",
      UTI: "public.plain-text",
    });
  };

  return (
    <View style={{ marginTop: 50 }}>
      <Button onPress={onShare} title="Share to WhatsApp" />
    </View>
  );
};

export default FileShareExample;
```

Apple Authentication

- Provides Apple authentication for iOS. It does not yet support Android or web.
- `Bunx expo install expo-apple-authentication || npx expo install expo-apple-authentication`

Blur View

- A React component that blurs everything underneath the view. Common usage of this is for navigation bars, tab bars, and modals.
- `Bunx expo install expo-blur || npx expo install expo-blur`

```
<BlurView intensity={100} style={styles.blurContainer}>  
  <Text style={styles.text}>{text}</Text>  
</BlurView>
```

Thank You !